Experiment 'p_6_2_e_remove_middle_seq' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_6_2_e_remove_middle_seq
Natural language statement: Write a method to remove the middle element of an array if the array length is odd, or the middle two elements if the length is even.
Method signature: p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)

Ensures

- `|result| == if |inputs| % 2 == 1 then |inputs| - 1 else |inputs| - 2`
- `|inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j] == inputs[j + 1]`
- `|inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2 - 1 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==> result[j] == inputs[j + 2]`

Requires

- `|inputs| >= 1`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to remove the middle element of an array if the array length is odd, or
    ↪ the middle two elements if the length is even.

The signature should be:

p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)

The method should respect the following contract:

requires |inputs| >= 1, ensures |result| == if |inputs| % 2 == 1 then |inputs| - 1 else |
    ↪ inputs| - 2, ensures |inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==>
    ↪ result[i] == inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j]
    ↪ == inputs[j + 1], ensures |inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2
    ↪ - 1 ==> result[i] == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==>
    ↪  result[j] == inputs[j + 2]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn0HMVXwONBbW8K0vMGOO6nHlYcpQ
Model: gpt-5.5-2026-04-23
Created at: 1780571516
Finish reason: stop
Usage: 606 tokens in, and 1186 tokens out

```
{
  if |inputs| % 2 == 1 {
    var m := |inputs| / 2;
    var newLen := |inputs| - 1;
    result := seq(newLen, k => if k < m then inputs[k] else inputs[k + 1]);
  } else {
    assert |inputs| >= 2;
    var m := |inputs| / 2;
    var newLen := |inputs| - 2;
    result := seq(newLen, k => if k < m - 1 then inputs[k] else inputs[k + 2]);
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_6_2_e_remove_middle_seq/program.dfy(10,51): Error: index out of range
   |
10 | result := seq(newLen, k => if k < m then inputs[k] else inputs[k + 1]);
   | ^

problem_data/p_6_2_e_remove_middle_seq/program.dfy(10,66): Error: index out of range
   |
10 | result := seq(newLen, k => if k < m then inputs[k] else inputs[k + 1]);
   | ^

problem_data/p_6_2_e_remove_middle_seq/program.dfy(15,55): Error: index out of range
   |
15 | result := seq(newLen, k => if k < m - 1 then inputs[k] else inputs[k + 2]);
   | ^

problem_data/p_6_2_e_remove_middle_seq/program.dfy(15,70): Error: index out of range
   |
15 | result := seq(newLen, k => if k < m - 1 then inputs[k] else inputs[k + 2]);
   | ^


Dafny program verifier finished with 1 verified, 4 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn0IEEY0on9StDifo05BeQwhaIC1A
Model: gpt-5.5-2026-04-23
Created at: 1780571570
Finish reason: stop
Usage: 1130 tokens in, and 7021 tokens out

```
{
  var n := |inputs|;
  if n % 2 == 1 {
    var m := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == m * 2 + 1;
    assert m * 2 == 2 * m;
    assert n == 2 * m + 1;
    assert 0 <= m;
    assert m + 1 <= n;

    var left := inputs[..m];
```

```
  var right := inputs[m + 1..];
  result := left + right;

  assert |left| == m;
  assert |right| == n - (m + 1);
  assert |result| == |left| + |right|;
  assert |result| == n - 1;

  forall i | 0 <= i < m
    ensures result[i] == inputs[i]
  {
    assert i < |left|;
    assert result[i] == left[i];
    assert left[i] == inputs[i];
  }

  forall j | m <= j < |result|
    ensures result[j] == inputs[j + 1]
  {
    assert |left| == m;
    assert j >= |left|;
    assert 0 <= j - m;
    assert j < n - 1;
    assert j - m < n - (m + 1);
    assert j - m < |right|;
    assert j - |left| == j - m;
    assert result[j] == right[j - |left|];
    assert result[j] == right[j - m];
    assert right[j - m] == inputs[(m + 1) + (j - m)];
    assert (m + 1) + (j - m) == j + 1;
  }
} else {
  var m := n / 2;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  assert n == (n / 2) * 2 + n % 2;
  assert n == m * 2;
  assert m * 2 == 2 * m;
  assert n == 2 * m;
  assert 0 <= m;
  if m < 1 {
    assert m == 0;
    assert n == 0;
    assert false;
  }
  assert m >= 1;
  assert m - 1 >= 0;
  assert m + 1 <= n;

  var left := inputs[..(m - 1)];
  var right := inputs[m + 1..];
```

```
    result := left + right;

    assert |left| == m - 1;
    assert |right| == n - (m + 1);
    assert |result| == |left| + |right|;
    assert |result| == n - 2;

    forall i | 0 <= i < m - 1
      ensures result[i] == inputs[i]
    {
      assert i < |left|;
      assert result[i] == left[i];
      assert left[i] == inputs[i];
    }

    forall j | m - 1 <= j < |result|
      ensures result[j] == inputs[j + 2]
    {
      assert |left| == m - 1;
      assert j >= |left|;
      assert 0 <= j - (m - 1);
      assert j < n - 2;
      assert j - (m - 1) < n - (m + 1);
      assert j - (m - 1) < |right|;
      assert j - |left| == j - (m - 1);
      assert result[j] == right[j - |left|];
      assert result[j] == right[j - (m - 1)];
      assert right[j - (m - 1)] == inputs[(m + 1) + (j - (m - 1))];
      assert (m + 1) + (j - (m - 1)) == j + 2;
    }
  }
}
```

Final Program

```
method p_6_2_e_remove_middle_seq(inputs: seq<int>) returns (result: seq<int>)
       requires |inputs| >= 1
       ensures |result| == if |inputs| % 2 == 1 then |inputs| - 1 else |inputs| - 2
       ensures |inputs| % 2 == 1 ==> forall i :: 0 <= i < |inputs| / 2 ==> result[i] ==
           ↪ inputs[i] && forall j :: |inputs| / 2 <= j < |result| ==> result[j] ==
           ↪ inputs[j + 1]
       ensures |inputs| % 2 == 0 ==> forall i :: 0 <= i < |inputs| / 2 - 1 ==> result[i]
           ↪ == inputs[i] && forall j :: |inputs| / 2 - 1 <= j < |result| ==> result[j]
           ↪ == inputs[j + 2]
{
  var n := |inputs|;
  if n % 2 == 1 {
    var m := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == m * 2 + 1;
```

```
  assert m * 2 == 2 * m;
  assert n == 2 * m + 1;
  assert 0 <= m;
  assert m + 1 <= n;

  var left := inputs[..m];
  var right := inputs[m + 1..];
  result := left + right;

  assert |left| == m;
  assert |right| == n - (m + 1);
  assert |result| == |left| + |right|;
  assert |result| == n - 1;

  forall i | 0 <= i < m
    ensures result[i] == inputs[i]
  {
    assert i < |left|;
    assert result[i] == left[i];
    assert left[i] == inputs[i];
  }

  forall j | m <= j < |result|
    ensures result[j] == inputs[j + 1]
  {
    assert |left| == m;
    assert j >= |left|;
    assert 0 <= j - m;
    assert j < n - 1;
    assert j - m < n - (m + 1);
    assert j - m < |right|;
    assert j - |left| == j - m;
    assert result[j] == right[j - |left|];
    assert result[j] == right[j - m];
    assert right[j - m] == inputs[(m + 1) + (j - m)];
    assert (m + 1) + (j - m) == j + 1;
  }
} else {
  var m := n / 2;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  assert n == (n / 2) * 2 + n % 2;
  assert n == m * 2;
  assert m * 2 == 2 * m;
  assert n == 2 * m;
  assert 0 <= m;
  if m < 1 {
    assert m == 0;
    assert n == 0;
    assert false;
  }
```

```
    assert m >= 1;
    assert m - 1 >= 0;
    assert m + 1 <= n;

    var left := inputs[..(m - 1)];
    var right := inputs[m + 1..];
    result := left + right;

    assert |left| == m - 1;
    assert |right| == n - (m + 1);
    assert |result| == |left| + |right|;
    assert |result| == n - 2;

    forall i | 0 <= i < m - 1
      ensures result[i] == inputs[i]
    {
      assert i < |left|;
      assert result[i] == left[i];
      assert left[i] == inputs[i];
    }

    forall j | m - 1 <= j < |result|
      ensures result[j] == inputs[j + 2]
    {
      assert |left| == m - 1;
      assert j >= |left|;
      assert 0 <= j - (m - 1);
      assert j < n - 2;
      assert j - (m - 1) < n - (m + 1);
      assert j - (m - 1) < |right|;
      assert j - |left| == j - (m - 1);
      assert result[j] == right[j - |left|];
      assert result[j] == right[j - (m - 1)];
      assert right[j - (m - 1)] == inputs[(m + 1) + (j - (m - 1))];
      assert (m + 1) + (j - (m - 1)) == j + 2;
    }
  }
}
```

Total Token Usage

Input tokens: 1736
Output tokens: 8207
Reasoning tokens: 6989
Sum of 'total tokens': 9943

Experiment Timings

Overall Experiment started at 1780571516459, ended at 1780571725372, lasting 208913ms (208.91 seconds)
Iteration #1 started at 1780571516461, ended at 1780571570640, lasting 54179ms (54.18 seconds)

Iteration #2 started at 1780571570650, ended at 1780571725372, lasting 154722ms (154.72 seconds)